Reporte Técnico de Actividades Práctico-Experimentales Nro. 005

1. Datos de Identificación del Estudiante y la Práctica

Nombre del estudiante(s)	Jaime A. Landazuri
Asignatura	Simulación
Ciclo	5
Unidad	U2: Generación de números pseudoaleatorios y generación de variables aleatorias
Resultado de aprendizaje de la unidad	R2. Interpreta los resultados de una Simulación, bajo los principios de solidaridad, transparencia, responsabilidad y honestidad.
Práctica Nro.	05
Título de la Práctica	Implementación y análisis del Generador Congruencial Lineal en Python y Java
Nombre del Docente	José O. Guamán Q.
Fecha	29 de octubre de 2026 (Jueves); 30 de octubre de 2026 (Viernes)
Horario	Jueves 10h30 – 11h30 Viernes 07h30 – 09h30
Lugar	Aula
Tiempo planificado en el Sílabo	3 horas

2. Objetivo(s) de la Práctica

1. Comprender el funcionamiento de un generador pseudoaleatorio mediante la implementación del método LCG (Generador Congruencial Lineal) en Python.
2. Implementar el mismo algoritmo en Java.
3. Comparar las secuencias generadas y analizar sus propiedades.

3. Materiales, Reactivos, Equipos y Herramientas

- Computador.
- Python (versión 3.14); biblioteca `matplotlib` para la generación de gráficos.
- Java / JDK (OpenJDK 27, compilación con `javac`, ejecución con `java`).
- Sistema operativo Linux.

4. Procedimiento / Metodología Ejecutada

Se siguió el procedimiento indicado en la guía APE Nro. 004 (Práctica 05):

1. Se seleccionaron los parámetros especificados por la guía: $a = 1664525$, $c = 1013904223$, $m = 2^{32}$ y semilla $= 12345$. La recursión implementada es $X_{n+1} = (a \cdot X_n + c) \bmod m$, con valores normalizados $U_n = X_n/m$.

2. Se implementó el generador en Python (archivo `lcg.py`) y se generaron los $10{,}000$ valores de la secuencia.
3. Se implementó el mismo algoritmo en Java (archivo `LCG.java`), con manipulación modular de enteros de 32 bits. El producto $a \cdot X_n$ excede el rango del tipo `long`, por lo que la multiplicación se descompuso en productos parciales de 16 bits (`xLoûyLo`, etc.) y se aplicó la máscara `& 0xFFFFFFFFL`, evitando el desbordamiento sin recurrir a `BigInteger`.
4. Se compararon las primeras 20 observaciones de ambos lenguajes (Sección 5).
5. Se graficó la secuencia: evolución de $U(n)$, diagrama de dispersión $U(n)$ vs. $U(n+1)$, histograma de frecuencias contra la uniforme ideal y resumen estadístico.
6. Se analizó el período: con $m = 2^{32}$ la verificación empírica no es iterable, por lo que la demostración se realizó con módulos pequeños ($m = 16$) contrastando la hipótesis de Hull–Dobell (con c coprimo con m y $a \equiv 1 \pmod 4$, cuando m es potencia de 2, el período es completo e igual a m). Se comprobó que c es impar y $a - 1 = 1664524 \equiv 0 \pmod 4$.
7. Se cambió la semilla a 54321 y se repitió la generación completa en ambos lenguajes.
8. Se compararon los resultados entre semillas y entre lenguajes mediante la exportación de archivos CSV idénticos en formato.

5. Resultados

5.1. Comparación de las primeras 20 observaciones (semilla 12345)

Ambos programas producen exactamente la misma secuencia; la columna “Idéntico?” verifica fila por fila tanto el valor crudo X_n (entero) como el normalizado U_n (10 decimales).

n	X_n (Python)	X_n (Java)	$U_n = X_n/m$	Idéntico?
1	87628868	87628868	0.0204026857	√
2	71072467	71072467	0.0165478482	√
3	2332836374	2332836374	0.5431557945	√
4	2726892157	2726892157	0.6349040561	√
5	3908547000	3908547000	0.9100295138	√
6	483019191	483019191	0.1124616691	√
7	2129828778	2129828778	0.4958894053	√
8	2355140353	2355140353	0.5483488443	√
9	2560230508	2560230508	0.5961001171	√
10	3364893915	3364893915	0.7834504161	√
11	171172990	171172990	0.0398543174	√
12	3194601925	3194601925	0.7438012224	√
13	4148119648	4148119648	0.9658093676	√
14	316399679	316399679	0.0736675409	√
15	3004788882	3004788882	0.6996069294	√
16	1976948425	1976948425	0.4602941743	√
17	1702883732	1702883732	0.3964835154	√
18	4121112547	4121112547	0.9595212869	√
19	1744294886	1744294886	0.4061253010	√
20	4092090893	4092090893	0.9527641565	√

5.2. Verificación de las secuencias completas (10.000 valores)

Se exportaron los 10.000 valores a CSV desde ambos lenguajes y se compararon de forma automatizada, tanto el valor crudo X_n como el normalizado U_n:

Semilla	Diferencias en primeras 20	Diferencias en los 10.000 valores	Resultado	
12345	0 / 20	0 / 10.000	√	
54321	0 / 20	0 / 10.000	√	

5.3. Experimento con cambio de semilla (54321)

Al repetir la generación con semilla 54321 se obtuvo una secuencia distinta (y también idéntica entre Python y Java). Las primeras observaciones contrastan así entre semillas:

n	X_n (semilla 12345)	U_n (semilla 12345)	X_n (semilla 54321)	U_n (semilla 54321)
1	87628868	0.0204026857	1238253532	0.2883033669
2	71072467	0.0165478482	1708513675	0.3977943386
3	2332836374	0.5431557945	3683344750	0.8575955289
4	2726892157	0.6349040561	1863498229	0.4338794921
5	3908547000	0.9100295138	2137457360	0.4976655729
6	483019191	0.1124616691	102299631	0.0238184889
7	2129828778	0.4958894053	3033777282	0.7063563173
8	2355140353	0.5483488443	4230855161	0.9850727303
9	2560230508	0.5961001171	1814797060	0.4225403676
10	3364893915	0.7834504161	1036872339	0.2414156541
11	171172990	0.0398543174	2695818966	0.6276692650
12	3194601925	0.7438012224	1801476157	0.4194388532
13	4148119648	0.9658093676	2976955512	0.6931264680
14	316399679	0.0736675409	2448971127	0.5701955238
15	3004788882	0.6996069294	4038571114	0.9403031119
16	1976948425	0.4602941743	1174362305	0.2734275314
17	1702883732	0.3964835154	849107756	0.1976983054
18	4121112547	0.9595212869	33496219	0.0077989462
19	1744294886	0.4061253010	3337365822	0.7770410324
20	4092090893	0.9527641565	1978253189	0.4605979633

5.4. Gráficos de la secuencia

La Figura 1 corresponde a la secuencia con semilla 12345 y la Figura 2 a la de semilla 54321. Cada figura muestra: (a) la evolución de $U(n)$ en las primeras 500 iteraciones; (b) el diagrama de dispersión $U(n)$ vs. $U(n+1)$, que permite observar la correlación serial de los pares consecutivos; (c) el histograma de frecuencias (100 intervalos) contrastado con la densidad de una uniforme ideal; y (d) el resumen estadístico.

El análisis de los cuatro paneles es el siguiente:

- **Evolución de $U(n)$ (panel a):** en ambas semillas la traza salta sin un patrón visible por todo el intervalo $[0, 1)$; no se observan tendencias, secciones planas ni "huecos" prolongados, lo que es el comportamiento esperado de una secuencia que llena el rango de manera homogénea.
- **Diagrama $U(n)$ vs. $U(n+1)$ (panel b):** los $9\,999$ pares consecutivos forman una nube que cubre el cuadrado $[0, 1]^2$ sin bandas ni franjas evidentes (la correlación serial visual es baja). Debe notarse que todo LCG tiene una estructura reticular subyacente: estados consecutivos caen sobre una malla determinista, y aquí esa malla tiene un espaciado tan fino (2^{32} estados) que resulta invisible a esta escala, pero es detectable con pruebas espectrales; esto es una limitación conocida del LCG, no un defecto de estos parámetros en particular.
- **Histograma (panel c):** con $10\,000$ valores y 100 intervalos, la frecuencia esperada por intervalo es $\approx 10\,000/100$ con densidad 1; todas las barras se mantienen en un rango cercano (aprox. entre 70 % y 130 % de la altura ideal), y ninguna región

del intervalo queda vacía o sobrecargada de forma sistemática en ninguna de las dos semillas.

- **Resumen estadístico (panel d):** con semilla 12345 se obtuvo $\bar{U} = 0{,}49877$ (teórico $0{,}5$; desv. de $-0{,}04\,\%$) y $s = 0{,}288132$ (teórico $1/\sqrt{12} \approx 0{,}288675$; desv. de $-0{,}19\,\%$), con $\text{mín} = 0{,}00006$ y $\text{máx} = 0{,}99993$. Con semilla 54321 los valores son prácticamente idénticos: $\bar{U} = 0{,}49887$ y $s = 0{,}289131$ (desv. de $+0{,}16\,\%$). Los datos se concentran en $[0, 1)$ en ambos casos.

En conjunto, los gráficos evidencian que el generador con estos parámetros produce secuencias *estadísticamente compatibles* con una $U(0, 1)$ a esta escala de muestra: media y desviación muy próximas a los valores teóricos, cobertura uniforme del intervalo y ausencia de correlación serial visible. No obstante, confirman también que "compatibilidad": no es "aleatoriedad verdadera": la secuencia es completamente determinista y reproducible.

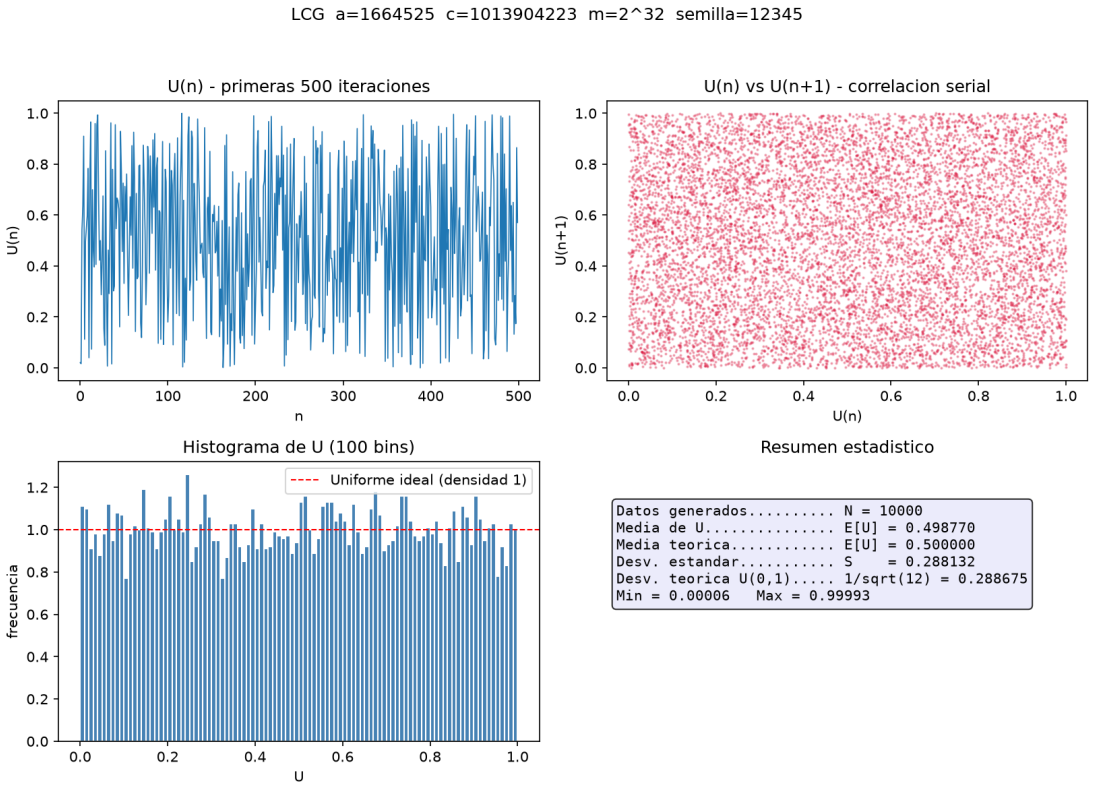


Figura 1: Análisis de la secuencia del LCG con semilla 12345 (mismo gráfico se generó para ambas implementaciones).

5.5. Análisis del período

La demostración empírica con módulos pequeños mostró tres escenarios:

a	c	m	Resultado empírico	
5	3	16	Período completo: el estado regresa a la semilla tras 16 iteraciones.	
5	2	16	Debe devolver un período de 8; el estado no recorre todo el rango.	
4	3	16	El estado entra en un ciclo que no contiene a la semilla; nunca regresa a X_0.	

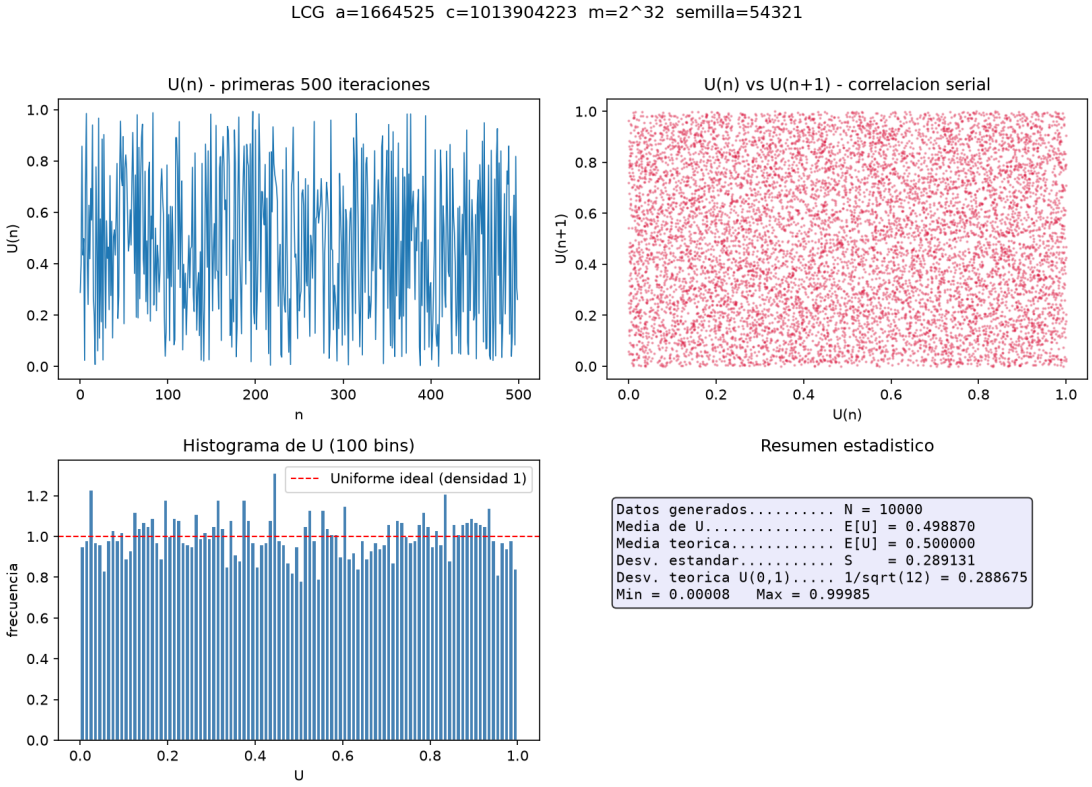


Figura 2: Análisis de la secuencia del LCG con semilla 54321.

Como $c = 1013904223$ es impar (coprimo con 2^{32}) y $a - 1 = 1664524 \equiv 0 \pmod 4$, los parámetros de la práctica cumplen la hipótesis de Hull–Dobell, por lo que el período teórico del generador es completo, igual a $m = 2^{32}$.

Conclusión del estudiante: un período completo de $2^{32} \approx 4{,}29 \times 10^9$ valores significa que la secuencia no repite estado hasta después de esa cantidad de iteraciones, y ese comportamiento no depende de la semilla elegida: cada semilla es simplemente un punto distinto dentro del mismo ciclo. Para esta práctica, con $n = 10\,000$, la muestra consume menos del $0{,}0003\,\%$ del ciclo, de modo que las $10\,000$ observaciones son todas distintas y la demostración empírica del período directo resulta impracticable; por eso se validó el criterio de Hull–Dobell con módulos pequeños. En la práctica de simulación esto tiene una consecuencia directa: la longitud del período garantiza que no habrá repetición de estados dentro del tamaño de muestra que usemos (siempre que $n \ll m$), pero un período largo es una condición *necesaria y no suficiente* de calidad: dice cuántos valores únicos entrega el generador, no qué tan bien distribuidos ni qué tan “impredecibles” son, que es lo que los gráficos y las pruebas estadísticas evalúan.

6. Preguntas de Control

1. **£Qué significa pseudoaleatorio?**

   Un número pseudoaleatorio es un valor producido por un algoritmo determinista que, aunque no es aleatorio en sentido estricto (el proceso es completamente predecible y reproducible), se comporta de manera estadística como si lo fuera: sus valores se distribuyen de forma uniforme en $[0, 1)$, sin correlaciones evidentes y sin un patrón detectable a escalas de interés. El prefijo “pseudo” refleja precisamente esa condición: son “aparentemente” aleatorios; dada la semilla y los parámetros, toda la secuencia queda determinada desde el inicio. En esta práctica se comprobó esa doble naturaleza: la secuencia parece al azar en sus gráficos y estadísticos, pero Python y Java produjeron exactamente los mismos $10\,000$ valores por construcción determinista.

2. **£Qué función cumple la semilla?**

   La semilla es el estado inicial X_0 de la recursión: el punto de partida del que derivan todos los demás valores. Cumple una doble función: (1) reproducibilidad, porque con la misma semilla y los mismos parámetros se obtiene siempre la misma secuencia (verificado aquí: $0/10\,000$ diferencias entre Python y Java para ambas semillas), lo que permite repetir experimentos de simulación y depurar código; y (2) variabilidad, porque una semilla distinta coloca al generador en un punto diferente del ciclo y produce un flujo distinto de valores (la secuencia con 54321 no comparte ningún valor ni posición con la de 12345). En simulación, fijar la semilla es la forma de hacer un experimento repetible, y variarla es la forma de obtener réplicas independientes.

3. **£Qué significa período?**

   El período es la longitud del ciclo de estados: el número de iteraciones a partir del cual el generador vuelve a un estado por el que ya pasó, y por tanto la secuencia se repite íntegramente. Es la propiedad más básica de calidad de un LCG, porque define cuántos valores distintos puede entregar como máximo. Aquí se distinguió entre el período empírico (observable recorriendo la secuencia, viable solo con módulos pequeños: con $a = 5$, $c = 3$, $m = 16$ se observó el período completo de 16 en la práctica) y el período teórico, demostrado con la hipótesis de Hull–Dobell: como c es coprimo con m y $a \equiv 1 \pmod 4$ siendo m potencia de 2, los parámetros de la práctica garantizan un período completo de $m = 2^{32}$.

4. **£Qué ocurre si se utilizan parámetros inadecuados?**

   El generador degrada de una o varias de estas formas, todas evidenciadas en los experimentos con módulos pequeños: (1) período incompleto o corto, si c no es coprimo con m o $a - 1$ no es divisible por todos los factores primos de m (con $a = 5$, $c = 2$, $m = 16$ el período fue 8 en lugar de 16); (2) ciclos que ni siquiera contienen a la semilla, cuando a y m no son coprimos (con $a = 4$, $c = 3$, $m = 16$ el estado entró en un ciclo ajeno a X_0); (3) correlación serial fuerte, que en el diagrama $U(n)$ vs. $U(n + 1)$ aparece como puntos concentrados en pocas líneas o bandas en lugar de una nube uniforme; y (4) histogramas no uniformes. En el límite, parámetros mal elegidos pueden reducir el generador a un ciclo de unos pocos valores, inútil para simular: el período $m/6$ del generador “malo” analizado en la práctica 06 es un ejemplo real de esta degradación.

5. **£Qué ventaja tiene implementar el mismo algoritmo en dos lenguajes?**

   Implementar el mismo LCG en Python y en Java y obtener las $10\,000$ observaciones idénticas sirve como verificación cruzada mutua: si los resultados coinciden,

la evidencia de que el algoritmo está bien implementado es mucho más fuerte que la de una sola implementación (un error en el manejo de enteros, típico de Java al exceder el rango de 32 bits, se habría detectado inmediatamente en la comparación). Además aporta: (1) portabilidad, porque demuestra que el generador es reproducible en plataformas y runtimes distintos, requisito habitual al distribuir una simulación; (2) comprensión conceptual, porque reproducir la aritmética modular en un lenguaje sin desbordamiento silencioso aclara mejor qué construcción está haciendo realmente el otro; y (3) una base común para futuras prácticas, donde los flujos pseudoaleatorios generados en un lenguaje puedan compararse con los del otro.

7. Conclusiones

1. La implementación del LCG con $a = 1664525$, $c = 1013904223$, $m = 2^{32}$ resultó reproducible al bit: Python y Java produjeron secuencias idénticas ($0/10\,000$ diferencias, tanto el valor crudo X_n como el normalizado U_n) para las semillas 12345 y 54321. La equivalencia exacta entre lenguajes valida ambas implementaciones y confirma que el algoritmo, no la plataforma, define la secuencia.

2. La secuencia depende exclusivamente de la semilla: cambiar de 12345 a 54321 generó un flujo completamente distinto (pero igualmente idéntico entre lenguajes), con estadísticos casi iguales ($\bar{U} \approx 0{,}499$ y $s \approx 0{,}289$ en ambos casos). Esto cumple la doble función buscada de la semilla: reproducibilidad del experimento y variabilidad entre experimentos.

3. Los gráficos muestran un generador estadísticamente sano a escala $n = 10\,000$: cobertura homogénea del intervalo, histograma compatible con la uniforme ideal, media y desviación estándar a menos del $0{,}2\,\%$ de los valores teóricos y sin correlación serial visible. Limitación conocida: todo LCG posee una estructura reticular subyacente invisible aquí, que solo pruebas espectrales detectarían.

4. El análisis del período demostró, vía Hull–Dobell (y de forma empírica con módulos pequeños), que los parámetros de la práctica entregan el período completo $m = 2^{32}$: cada semilla es un punto dentro de un solo ciclo de $\approx 4{,}29 \times 10^9$ valores. Con $n = 10\,000$ la muestra usa una fracción mínima del ciclo, de modo que ninguna observación se repite dentro del experimento.

5. La condición de período completo y la calidad estadística observada hacen que este LCG sea una base adecuada para las prácticas siguientes de la asignatura, siempre con la advertencia conceptual de que “pseudoaleatorio” implica determinismo: la “aleatoriedad” es una propiedad estadística de la secuencia, no del proceso que la genera.

8. Recomendaciones

- Documentar siempre la semilla junto con los resultados de una simulación: sin ella los resultados no son reproducibles y no pueden contrastarse con réplicas.

- Antes de usar cualquier LCG “casero” en simulaciones serias, verificar el criterio de Hull–Dobell para garantizar período completo y, si el tamaño de muestra es grande, someter el generador a pruebas estadísticas (Chi-cuadrado, Kolmogorov–Smirnov, pruebas de series) como las aplicadas en la práctica 06.

- Para aplicaciones con requisitos de calidad elevados (criptografía, simulación de eventos discretos de larga duración), preferir generadores con período mayor y

mejor comportamiento espectral (Mersenne Twister, PCG, xorshift), reservando el LCG clásico para docencia y prototipos.

- En Java, encapsular la aritmética modular de 32 bits en una función probada (la descomposición en productos de 16 bits aquí implementada), ya que el producto directo $a \cdot X_n$ excede el rango de `long` para valores intermedios altamente construidos y es fuente típica de errores silenciosos.

9. Declaración de uso de Inteligencia Artificial

En este trabajo se usaron herramientas de inteligencia artificial como apoyo, no como sustituto del trabajo propio. Primero se muestra un prompt representativo de la forma en que se usó la herramienta y, a continuación, la justificación de su uso.

9.1. Prompt utilizado

El estudiante partió siempre de contenido ya elaborado (código, ejecuciones y resultados propios) y pidió mejorarlo, no generarlo desde cero. El siguiente prompt resume la instrucción general empleada (es representativo, no una transcripción literal de todas las interacciones):

> *Implementé el LCG de la práctica 05 ($a = 1664525$, $c = 1013904223$, $m = 2^{32}$, semilla 12345) en Python y en Java, y ya ejecuté ambos programas con sus $10\,000$ valores, las gráficas y los CSV. Necesito que me ayudes a **mejorar** lo que ya tengo, no a hacerlo desde cero:*
>
> *1. Verificación del código: revisa la descomposición en productos parciales de 16 bits que usé en Java para la multiplicación modular sin desbordamiento, y dime si la implementación es correcta.*
>
> *2. Redacción: completa el esqueleto de mi informe LaTeX con el análisis de las gráficas, las preguntas de control y las conclusiones, a partir de mis tablas, mis figuras y mis resultados reales, sin inventar números.*
>
> *3. Formato: corrige el código LaTeX (tablas,*
> *input de archivos generados y referencias) para que compile en Overleaf sin superarposciones.*
>
> *Reglas: no inventes datos, cifras ni referencias; todo debe salir de mis archivos de resultados. Si algo no puedes verificarlo, daímelo.*

9.2. Justificación del uso

Herramientas utilizadas.

- **GLM-5.3-flash**, mediante un agente de código en entorno local.

Apoyo recibido del modelo.

- **Verificación del código:** revisión de la implementación de la multiplicación modular de 32 bits en Java y de la equivalencia estructural con la versión en Python.
- **Redacción:** elaboración del esqueleto del informe y redacción del análisis de resultados, preguntas de control y conclusiones a partir de las tablas, gráficas y CSV generados por las ejecuciones del estudiante.
- **Formato:** escritura y corrección del código LaTeX, generación de los fragmentos de tabla desde los CSV de resultados y compilación del PDF.

Lo que no se hizo con IA.

- La implementación por primera vez del algoritmo, la elección de los parámetros de la guía, las ejecuciones de los dos programas y la producción de los CSV y de las figuras son trabajo propio del estudiante; los modelos solo trabajaron sobre esos artefactos ya existentes.
- Todas las cifras y comparaciones del informe provienen de los archivos de resultados generados por las ejecuciones del estudiante; ningún valor fue hipotético o simulado mentalmente por el modelo.

Alcance y verificación: En este informe, como en las prácticas de la unidad anterior, la inteligencia artificial no se usó para elaborar el informe completo ni para generar resultados: se usó como herramienta de apoyo para redacción, verificación y formato. El estudiante revisó los textos, comprobó que las cifras y los comentarios coincidieran con sus propios resultados y es responsable del contenido final. Dado que las herramientas de IA pueden cometer errores, las afirmaciones del período y de la aritmética modular se contrastaron con la guía de la práctica y con la bibliografía citada.

10. Bibliografía / Referencias

- Guamán, J. O. (2026). *Guía de Actividades Práctico-Experimentales Nro. 004, Práctica 05: Implementación y análisis del Generador Congruencial Lineal en Python y Java*. FEIRNNR, Carrera de Computación.
- Diapositivas de la Semana 5 (material de clase, Simulación U2).

11. Anexos

- `lcg.py`: implementación del LCG en Python (genera 10.000 valores, CSV, gráficos y análisis de período).
- `LCG.java`: implementación espejo del algoritmo en Java.
- `resultados/python_primeras20_*.csv` y `resultados/java_primeras20_*.csv`: primeras 20 observaciones por lenguaje y semilla.
- `resultados/python_10000_*.csv` y `resultados/java_10000_*.csv`: secuencias completas de 10.000 valores por lenguaje y semilla.
- `lcg_seed12345.png` y `lcg_seed54321.png`: gráficos de la secuencia.